

Git & Markdown — os comandos que você vai usar sempre

Isso não substitui a Aula 29 (Git e GitHub para QA) — é uma folha de consulta rápida pra você lembrar a sintaxe e a ordem dos comandos sem precisar reabrir o slide toda vez.

arquivo.md

Extensão de arquivos escritos em **Markdown** — uma linguagem de formatação em texto puro. Em vez de clicar em "negrito" ou "título" como no Word, você escreve símbolos (#, *, -) que viram formatação quando o arquivo é exibido. É o padrão de documentação no GitHub: todo repositório tem um **README.md**, a primeira coisa que aparece quando alguém abre o projeto — inclusive um recrutador olhando seu portfólio.

```
# Título principal
## Subtítulo
**negrito** *itálico*
- item de lista
[texto do link](https://...)
```

1 **git clone**

Baixa uma cópia completa de um repositório remoto (ex: do GitHub) pra sua máquina, com todo o histórico de commits incluído. É o primeiro comando que você roda ao começar a trabalhar num projeto que já existe — normalmente só uma vez.

```
git clone https://github.com/seu-usuario/portfolio-qa.git
```

2 **git add**

Prepara ("adiciona ao stage") os arquivos que você mudou pra entrarem no próximo commit. O Git não salva tudo automaticamente — você escolhe explicitamente o que vai entrar na "foto" daquele momento.

```
git add relatorio-de-teste.md
```

2b **git add .**

Mesma ideia, mas o ponto (.) significa "tudo que mudou nesta pasta e nas subpastas". É o jeito mais comum de usar no dia a dia, quando você quer incluir todas as suas alterações de uma vez.

```
git add .
```



Cuidado: `git add .` inclui TUDO que não estiver no `.gitignore` — inclusive arquivo que você esqueceu de excluir (como senhas num `.env`). Rode `git status` antes de commitar, pra ver exatamente o que vai entrar.

3 `git commit`

Salva, no seu histórico local, tudo que foi preparado com `git add` — como uma "foto" daquele momento do projeto, junto com uma mensagem explicando o que mudou. Cada commit é um ponto ao qual você pode voltar depois, se precisar.

```
git commit -m "Adiciona plano de teste da Aula 13"
```

4 `git push`

Envia os commits que você fez localmente pro repositório remoto (GitHub), tornando-os visíveis pra qualquer pessoa que acesse o projeto — inclusive um recrutador olhando seu portfólio.

```
git push
```

5 `git pull`

Busca e traz pro seu computador as mudanças que foram enviadas pro repositório remoto (por outra pessoa, ou por você mesma de outra máquina). É o oposto do push: em vez de mandar, você recebe.

```
git pull
```

→ Ordem típica de uso

`git clone` → `editar arquivos` → `git add .` → `git commit -m "..."` → `git push`

git clone você faz uma vez só, no começo. **add → commit → push** você repete toda vez que progride no projeto. **git pull** você roda quando quiser buscar mudanças de outra máquina ou de outra pessoa antes de continuar.

💡 Analogia rápida

add = separar as fotos que você quer guardar do álbum. **commit** = colar essas fotos no álbum com uma legenda. **push** = mandar o álbum pra nuvem, onde todo mundo pode ver.

🔍 Comando bônus

git status — mostra o que mudou, o que já está no stage (add) e o que ainda não. Rode sempre antes de add e de commit, pra não errar o que vai subir.